

Design and Analysis of Algorithms

- *Course Code:* BCSE204L
- *Course Type:* Theory (ETH)
- *Slot:* A1+TA1 & & A2+TA2
- *Class ID:* VL2023240500901
VL2023240500902

A1+TA1

Day	Start	End
Monday	08:00	08:50
Wednesday	09:00	09:50
Friday	10:00	10:50

A2+TA2

Day	Start	End
Monday	14:00	14:50
Wednesday	15:00	15:50
Friday	16:00	16:50

Syllabus- Module 2

Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
----------	---	----------

Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP-

Backtracking: N-Queens problem, Subset Sum, Graph Coloring-

Branch & Bound: LIFO-BB and FIFO BB methods: **Job Selection problem**, 0-1 Knapsack Problem

Branch and Bound (BB)

Here are some key points about Branch and Bound:

- **Optimization Problems:** BB is particularly useful for solving optimization problems, where the goal is to find the best solution among a set of feasible solutions. These problems often involve minimizing or maximizing an objective function subject to certain constraints.
- **Minimization vs. Maximization:** BB is typically applied to minimization problems. However, it is possible to adapt maximization problems into minimization ones. For instance, converting a maximization problem into a minimization one by taking the negative of the objective function.
- **Graph Traversal:** BB traverses the state space tree using a **breadth-first search** (BFS) approach rather than depth-first search (DFS) used by backtracking algorithms. This allows BB to systematically explore the entire state space in an organized manner.
- **Types of BB:**
 - **LIFO BB:** Implemented using a stack data structure, LIFO (Last-In-First-Out) BB explores nodes in a depth-first manner.
 - **FIFO BB:** Implemented using a queue data structure, FIFO (First-In-First-Out) BB explores nodes in a breadth-first manner.
 - **LC BB:** Implemented using a least-cost strategy, LC (Least Cost) BB prioritizes exploring nodes with the lowest estimated cost or bound.

Branch and Bound (BB) @ applications

- Job Selection problem
- 0/1 knapsack problem
- Travelling sales-person problem

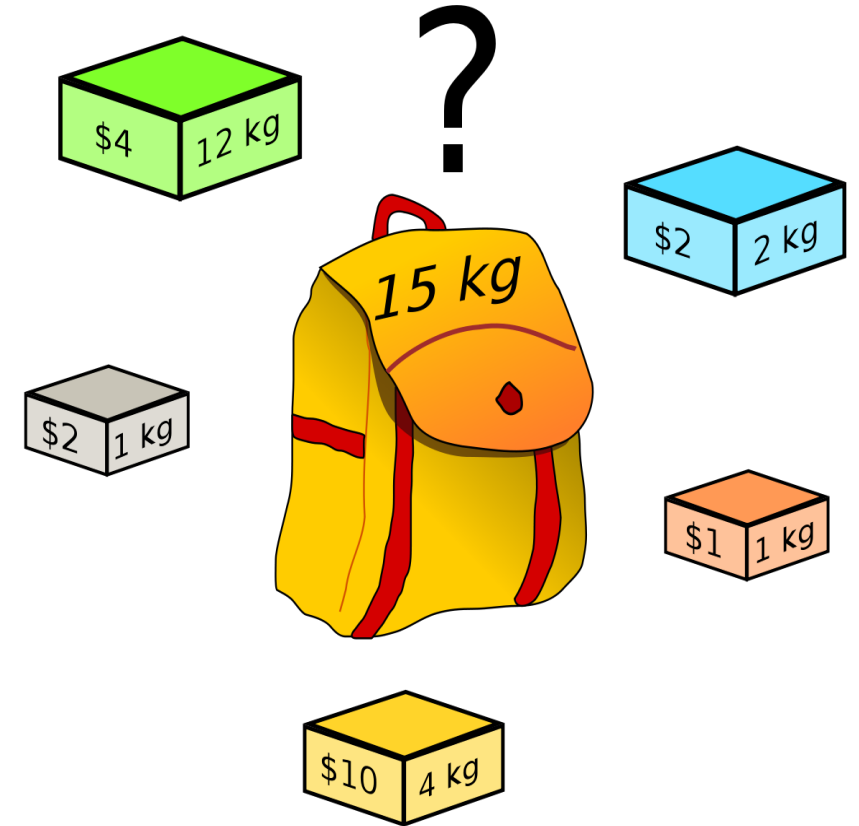
0/1 knapsack problem

- Given two integer arrays
 - $val[0..n-1]$ and
 - $wt[0..n-1]$

which represent values and weights associated with n items respectively.

Also given an integer W which represents knapsack capacity,

find out the maximum value subset of $val[]$ such that sum of the weights of this subset is smaller than or equal to W .



0/1 knapsack problem:

Brute force approach (A naive recursive implementation)

- Consider all subsets of items and calculate the total weight and value of all subsets. Consider the only subsets whose total weight is smaller than W .
- From all such subsets, pick the maximum value subset.

0-1 Knapsack Problem

value[] = {60, 100, 120};
weight[] = {10, 20, 30};
 $W = 50$;

Solution: 220

Weight = 10; Value = 60;
Weight = 20; Value = 100;
Weight = 30; Value = 120;
Weight = (20+10); Value = (100+60);
Weight = (30+10); Value = (120+60);
Weight = (30+20); Value = (120+100);
Weight = (30+20+10) > 50

Brute force approach

value[] = {60, 100, 120};
weight[] = {10, 20, 30};
C = 50;

Solution: 220

Weight = 10; Value = 60;
Weight = 20; Value = 100;
Weight = 30; Value = 120;
Weight = (20+10); Value = (100+60);
Weight = (30+10); Value = (120+60);
Weight = (30+20); Value = (120+100);
Weight = (30+20+10) > 50

```
int getMaxVal(int n , int w[] , int v[] , int c , int i)
{
    int vin , vex ;
    if(i==n || c<=0){
        return 0 ;
    }

    if(w[i] < c){
        vin = getMaxVal(n , w , v , c-w[i] , i+1)+v[i] ;
        vex = getMaxVal(n , w , v , c , i+1) ;
        return max(vin , vex) ;
    }
    else{
        vex =getMaxVal(n , w , v , c , i+1) ;
        return vex ;
    }
}
```

Brute force approach

```
algorithm knapSack(W, wt, val, n){
    # Base Case
    if n == 0 or W == 0:
        return 0

    if (wt[n-1] > W):
        return knapSack(W, wt, val, n-1)
    else:
        return max(
            val[n-1] + knapSack(W-wt[n-1], wt, val, n-1),
            knapSack(W, wt, val, n-1))
}
```

Time Complexity: $O(2^n)$.

As there are redundant subproblems.

0/1 Knapsack problem using Branch and Bound

- Consider the 0/1 knapsack problem with n objects and a total weight W . We define the **upper bound** (UB) to be the summation of $v_i * x_i$ (where v_i denotes the value of that object, and x_i is a binary value indicating whether the object is to be included or not), such that the total weights of the included objects are less than W .
- The **cost function** is defined as the summation of $v_i * f_i$, where f_i indicates the fraction of the object that is **to be included**, such that the total value is the maximum that can be obtained which is less than or equal to W . Although fractions are used in this calculation, they are not included in the final solution.

$$f_i = V_i / w_i$$

upper bound (UB)

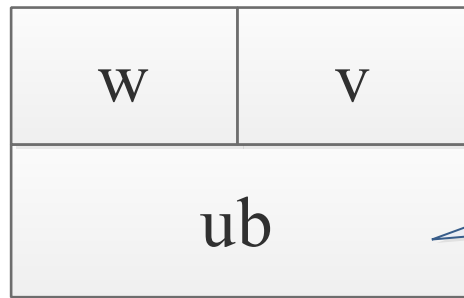
$$Ub = v + (W - w) \frac{V_{i+1}}{w_{i+1}}$$

Steps

1. Calculate the cost function and the upper bound for the two children of each node. Here, the $(i + 1)$ th level indicates whether the i th object is to be included or not.
2. If the cost function for a given node is greater than the upper bound, then the node need not be explored further. Hence, we can prune this node. Otherwise, calculate the upper bound for this node. If this value is less than UB, then replace the value of U with this value. Then, prune all unexplored nodes which have a cost function greater than this value.
3. The next node to be checked after reaching all nodes in a particular level will be the one with the least cost function value among the unexplored nodes.
4. While including an object, one needs to check whether adding the object crossed the threshold. If it does, one has reached the terminal point in that branch, and all the succeeding objects will not be included.

0/1 knapsack problem

Branch and Bound (BB)



$$Ub = v + (W - w) \frac{V_{i+1}}{W_{i+1}}$$

$$\frac{V_{i+1}}{W_{i+1}}$$

Value per unit weight of the next article

$w \rightarrow$ Considered items weight sum

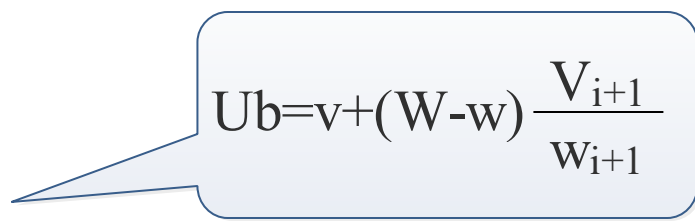
$v \rightarrow$ Considered items value sum

$W \rightarrow$ Knapsack Capacity

Item	Weight	Value	Values/Wight
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

$W=10$

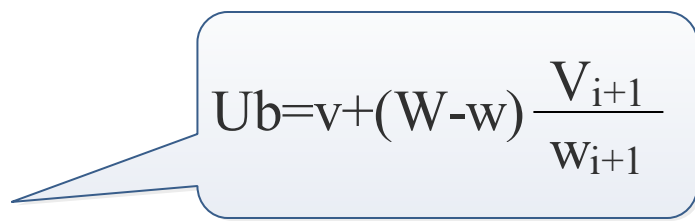
w	v
ub	



$$Ub = v + (W - w) \frac{V_{i+1}}{w_{i+1}}$$

Item	Weight	Value	Values/Wight
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

w	v
ub	



$$Ub = v + (W - w) \frac{V_{i+1}}{w_{i+1}}$$

Item	Weight	Value	Values/Wight
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

Time complexity and derive the recurrence relation

1. Sorting Items: Sorting the items based on their value-to-weight ratios takes $O(n \log n)$ time.

2. Branch and Bound Algorithm:

1. At each level of the search tree, we examine two children nodes (one including the current item and one excluding it). So, the number of nodes at each level is bounded by 2.
2. In the worst case, the algorithm explores all possible combinations of items, leading to a binary tree with a depth of n (the number of items).
3. At each node, we compute the upper bound and check whether to prune the node or not, which can be done in constant time or linear time, depending on the computation required.
4. Therefore, the time complexity of the Branch and Bound algorithm is exponential in the worst case, but typically much lower due to pruning.

3. Overall Time Complexity: Considering the sorting and the Branch and Bound algorithm, the overall time complexity can be approximated as $O(n \log n) + O(2^n)$.

$$O(n \log n) + O(2^n)$$

Note: The time complexity of the 0/1 Knapsack Problem using Branch and Bound depends on several factors, including the number of items (n), the capacity of the knapsack (W), and the distribution of item values and weights. In the worst case, the time complexity can be exponential, but the Branch and Bound algorithm often reduces the search space significantly, leading to better performance compared to brute-force methods.

Compare the procedures for solving the 0/1 Knapsack problem using Brute Force, Dynamic Programming, and Branch and Bound approaches.

Approach	Time Complexity	Procedure	Advantages	Disadvantages
Brute Force	$O(2^n)$	Generate all possible subsets, calculate value	-Simple implementation -Guaranteed to find optimal solution for small n	-Exponential time complexity - Impractical for large instances
Dynamic Programming	$O(nW)$	Build table iteratively, fill in optimal values	- Optimally solves problem for all instances. - Polynomial time complexity.	-Requires extra space for table. - Inefficient for large capacities or n
Branch and Bound	$O(n \log n) + O(2^n)$	Explore solution space, pruning as necessary	- Often performs better than brute force. - Can handle larger instances better than brute force	- Exponential time complexity in worst case. - Complexity depends on effectiveness of pruning

Compare the procedures for solving the 0/1 Knapsack problem using Brute Force, Dynamic Programming, and Branch and Bound approaches.

- The time complexity of 0/1 knapsack problem in brute force approach is $O(2^n)$
- Time complexity of 0/1 knapsack problem in Dynamic programming is $O(NW)$.
- Time complexity 0/1 Knapsack problem in Branch and bound is $O(n \log n) + O(2^n)$.
- Use Brute Force for small problem sizes or when simplicity is more important than efficiency.
- Use Dynamic Programming when the knapsack capacity is relatively small and memory resources are available.
- Use Branch and Bound for large problem sizes where efficient pruning can significantly reduce the search space and improve performance.

0/1 Knapsack problem using Branch and Bound

- Consider the problem with $n = 4$, $V = \{10, 10, 12, 18\}$, $w = \{2, 4, 6, 9\}$ and $W = 15$.

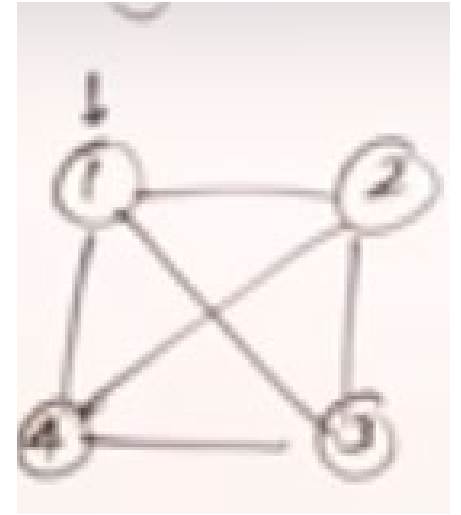
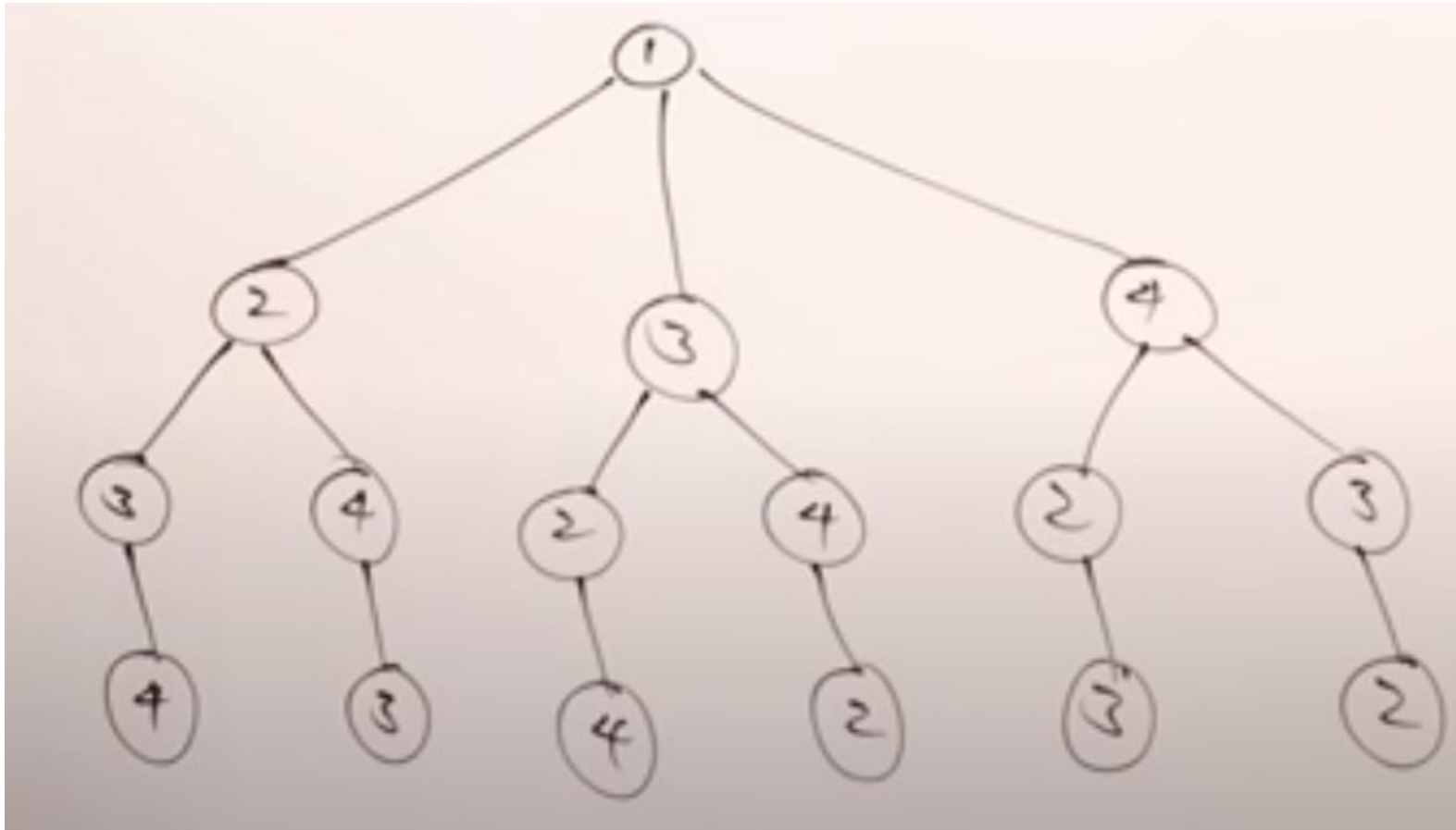
Item	Weight	Value	V/W
1	2	10	5
2	4	10	2.5
3	6	12	2
4	9	18	2

Branch and Bound (BB) @ applications

Travelling sales-person problem

Travelling sales person problem

State Space Tree



Travelling sales person problem using BB

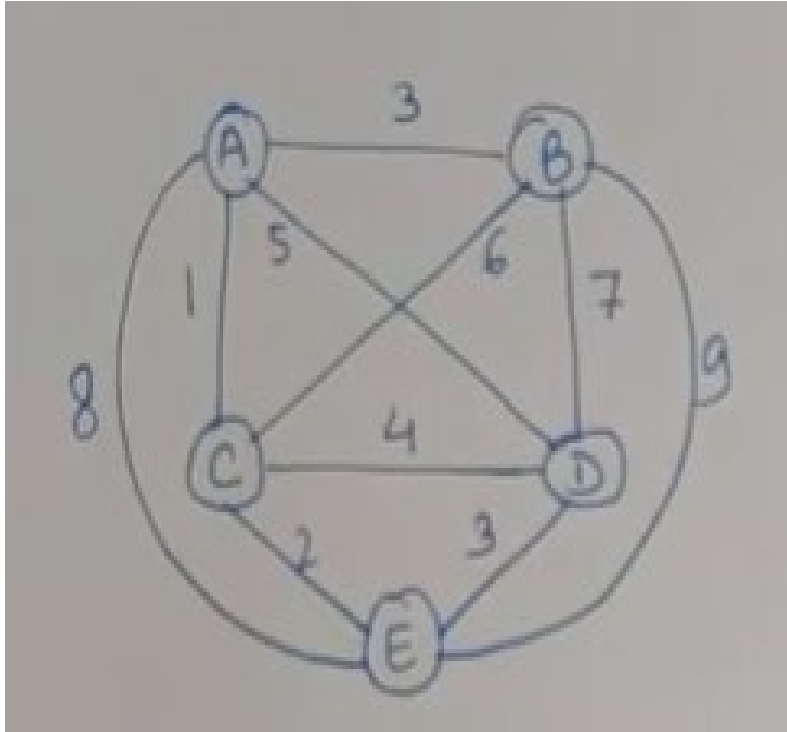
Reduced Matrix: Matrix M is called reduced matrix if each of its row and column has at least one zero entry or entire row or entire column has ∞ value.

Let M represents the distance matrix of 5 cities.

M can be reduced as follow:

$$M_{\text{RowRed}} = \{M_{ij} - \min \{M_{ij} \mid 1 \leq j \leq n, \text{ and } M_{ij} < \infty\}\}$$

Travelling sales person problem using BB



	A	B	C	D	E	
A	∞	3	1	5	8	1
B	3	∞	6	7	9	3
C	1	6	∞	4	2	1
D	5	7	4	∞	3	3
E	8	9	2	3	∞	2
						10

After row reduction :-

	A	B	C	D	E	
A	∞	2	0	4	7	
B	0	∞	3	4	6	
C	0	5	∞	3	1	
D	2	4	1	∞	0	
E	6	7	0	1	∞	
		2		1		

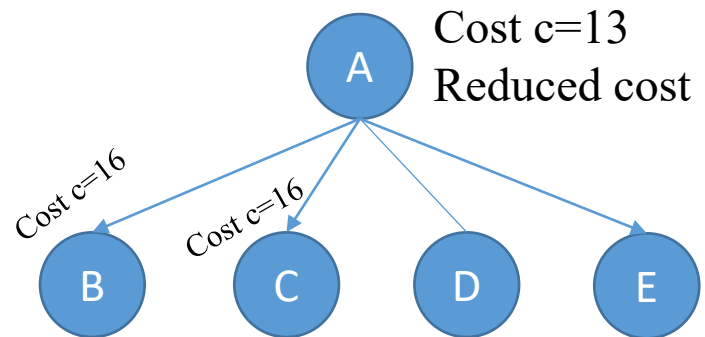
Total cost of reduction
 $= (10 + (2 + 1))$
 $= 10 + 3$
 $= 13$

	A	B	C	D	E
A	∞	0	0	3	7
B	0	∞	3	3	6
C	0	3	∞	2	1
D	2	2	1	∞	0
E	6	5	0	0	∞

reduced matrix

reduced matrix

	A	B	C	D	E
A	∞	0	0	3	7
B	0	∞	3	3	6
C	0	3	∞	2	1
D	2	2	1	∞	0
E	6	5	0	0	∞



$A \rightarrow B$

$A \rightarrow B$ col

Rw $\times$ (B, A)

	A	B	C	D	E
A	∞	∞	∞	∞	∞
B	∞	∞	3	3	6
C	0	∞	∞	2	1
D	2	∞	1	∞	0
E	6	∞	0	0	∞

3

∞	∞	∞	∞	∞
∞	∞	0	0	3
0	∞	∞	2	1
2	∞	1	∞	0
6	∞	0	0	∞

cost of reduction = 3

$$C(A \rightarrow B) = 13 + 3 + 0 = 16$$

$A \rightarrow C$

$A \rightarrow C$

	A	B	C	D	E
A	∞	∞	∞	∞	∞
B	0	∞	∞	3	6
C	∞	3	∞	2	1
D	2	2	∞	∞	0
E	6	5	∞	0	∞

2

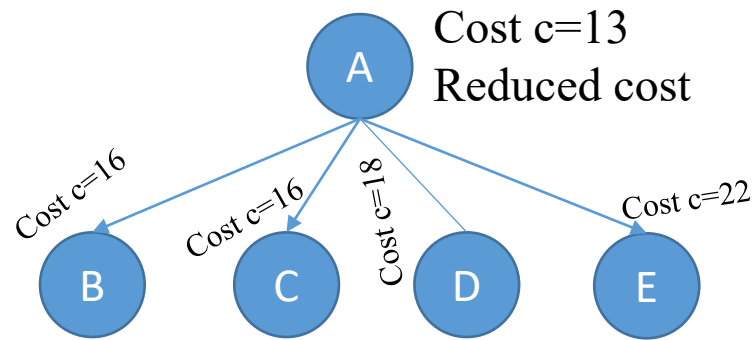
∞	∞	∞	∞	∞
0	∞	∞	3	6
∞	2	∞	1	0
2	0	∞	∞	0
6	3	∞	0	∞

cost of reduction = $(1+2) = 3$

$$C(A \rightarrow C) = 13 + 3 + 0 = 16$$

	A	B	C	D	E
A	∞	0	0	3	7
B	0	∞	3	3	6
C	0	3	∞	2	1
D	2	2	1	∞	0
E	6	5	0	0	∞

reduced matrix



$A \rightarrow D$

	A	B	C	D	E
A	∞	∞	∞	∞	∞
B	0	∞	3	∞	6
C	0	3	∞	∞	1
D	∞	2	1	∞	0
E	6	5	0	∞	∞

2

	A	B	C	D	E
A	∞	∞	∞	∞	∞
B	0	∞	3	∞	6
C	0	1	∞	∞	1
D	∞	0	1	∞	0
E	6	3	0	∞	∞

cost of reduction
= 2

$$C(A \rightarrow D) = 13 + 2 + 3 = 18$$

$A \rightarrow E$

	A	B	C	D	E
A	∞	∞	∞	∞	∞
B	0	∞	3	3	∞
C	0	3	∞	2	∞
D	2	2	1	∞	∞
E	∞	5	0	0	∞

1

	A	B	C	D	E
A	∞	∞	∞	∞	∞
B	0	∞	3	3	∞
C	0	2	∞	2	∞
D	1	0	0	∞	∞
E	∞	4	0	0	∞

$$C(A \rightarrow E) = 13 + 2 + 7 = 22$$

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 0 & 0 & 3 \\ 0 & \infty & \infty & 2 & 1 \\ 2 & \infty & 1 & \infty & 0 \\ 6 & \infty & 0 & 0 & \infty \end{bmatrix}$$

$A \rightarrow B \rightarrow C$

$A \rightarrow B \rightarrow C$

	A	B	C	D	E
A	∞	∞	∞	∞	∞
B	∞	∞	∞	∞	∞
C	∞	∞	∞	2	1
D	2	∞	∞	∞	0
E	6	∞	∞	0	∞

2

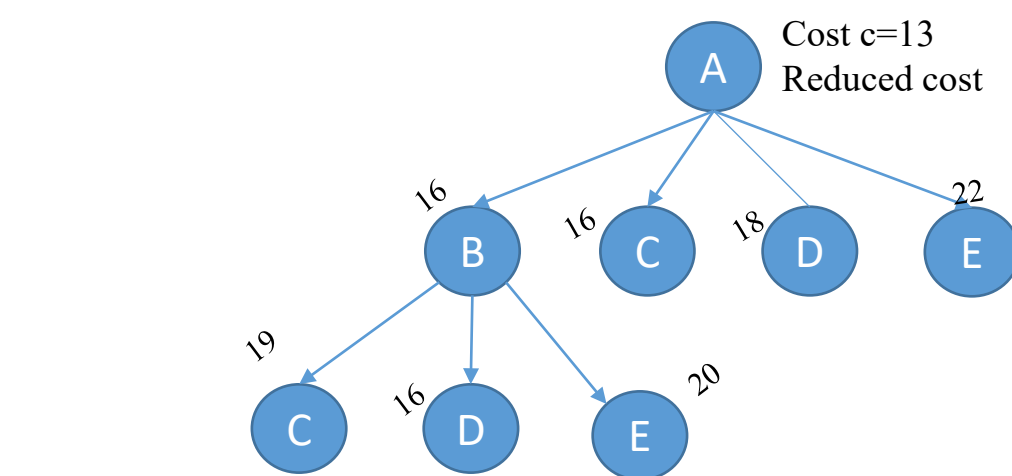
$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 1 & 0 \\ 0 & \infty & \infty & \infty & 0 \\ 4 & \infty & \infty & 0 & \infty \end{bmatrix}$$

$A \rightarrow B \rightarrow E$

$A \rightarrow B \rightarrow E$

	A	B	C	D	E
A	∞	∞	∞	∞	∞
B	∞	∞	∞	∞	∞
C	∞	∞	∞	2	∞
D	2	∞	1	∞	∞
E	∞	∞	0	0	∞

Dr. Venkata Phanikrishna B, S



$$\text{Cost}(A-B-C) = 16 + 3 + 0 = 19$$

$A \rightarrow B \rightarrow D$

$A \rightarrow B \rightarrow D$

	A	B	C	D	E
A	∞	∞	∞	∞	∞
B	∞	∞	∞	∞	∞
C	∞	∞	1	∞	0
D	6	∞	0	∞	∞

Cost $(A-B-D) = 16 + 0 = 16$

$$\text{Cost}(A-B-E) = 16 + 1 + 3 = 20$$

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & 1 \\ \infty & \infty & 1 & \infty & 0 \\ 6 & \infty & 0 & \infty & \infty \end{bmatrix}$$

A → B → D → C

A → B → D → E Row

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 1 \\ \infty & \infty & \infty & \infty & \infty \\ 6 & \infty & \infty & \infty & \infty \end{bmatrix}$$

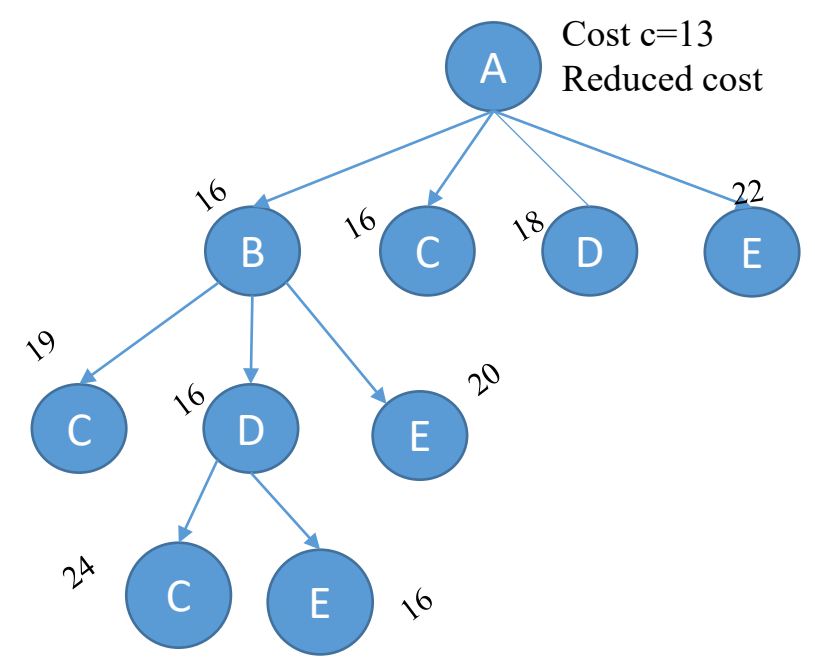
$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & \infty \end{bmatrix}$$

A → B → D → E

A → B → D → E

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 0 & \infty & \infty \end{bmatrix}$$

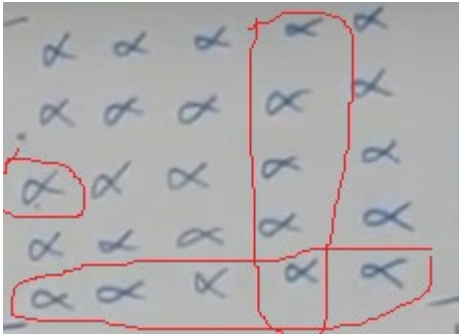
cost = 16 + 0 = 16



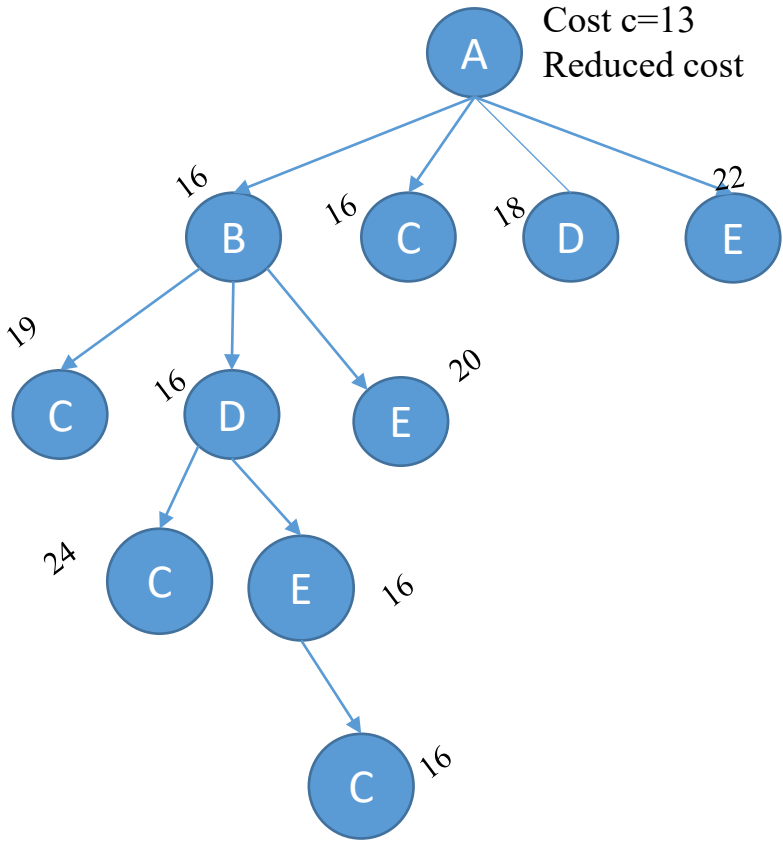
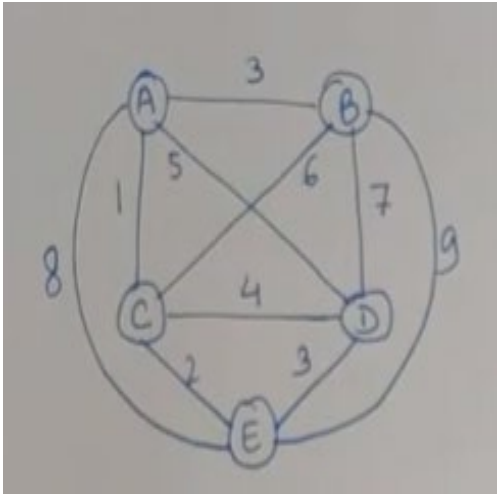
$$\text{Cost} = 16 + 7 + 1 = 24$$



$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$

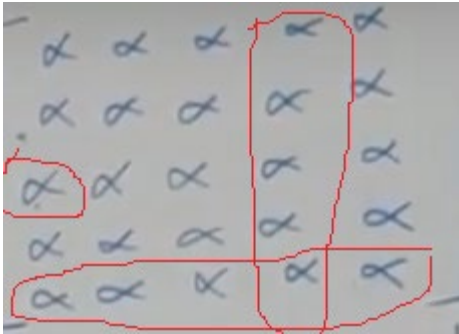


$$\text{Cost} = 16 + 0 + 0 = 16$$

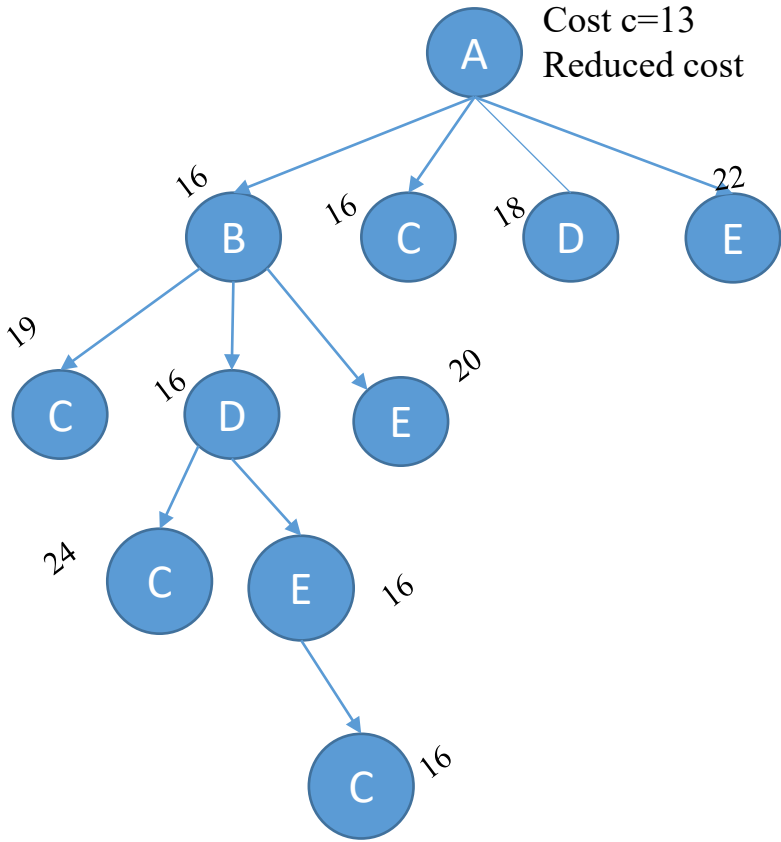
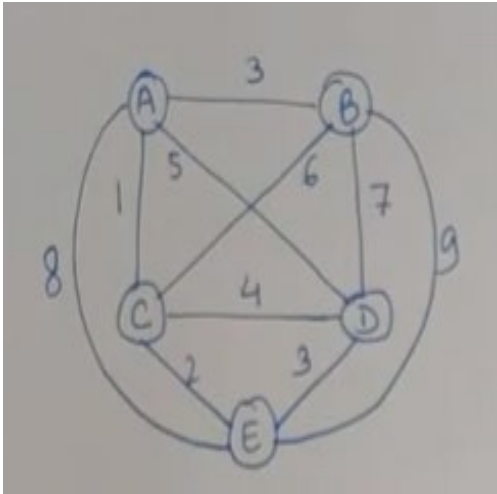




$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$



$$\text{Cost} = 16 + 0 + 0 = 16$$



Any

Question



PresenterMedia

Thank You!

**FOR YOUR
ATTENTION**

